Automating with Bash

NUnix · L3 MIAGE · CM3 · 90 minutes

ISTIC · University of Rennes 2026–2027



Louis Ledoux

Writing Bash scripts

Université de Rennes
Louis Ledoux

A support team has several log files. We will build a script that counts matching lines and saves a report.

1. Scripts, arguments and variables
2. Tests and decisions
3. Loops and counters
4. Functions and debugging
5. Build a log report

1. Scripts, arguments and variables

How can a script use a filename we type or pass as an argument?

A first script

Université de Rennes
Louis Ledoux

Save these commands in `hello.sh`:

```
#!/bin/bash
# A first script
echo "Hello, MIAGE"
```

- `#!/bin/bash` selects Bash for direct execution: the **shebang**.
- `#` begins a comment in this position.
- Commands run in order. A newline or `;` separates them.

Two ways to run a script

Université de Rennes
Louis Ledoux

Invoke Bash

```
bash hello.sh
```

Bash reads the file.
It needs **read** access to it.

Execute the file

```
chmod u+x hello.sh
./hello.sh
```

Requires **execute** permission and a usable interpreter.

Assigning and reading a variable

Université de Rennes
Louis Ledoux

```
filename="app.log"
echo "$filename"
filename="worker.log"
echo "Selected file: $filename"
```

`filename=value` stores a value. **No spaces around =.**
`$filename` reads that value. Quote it: `"$filename"`.

Asking the user with read

Université de Rennes
Louis Ledoux

```
read -r -p "File to inspect: " filename
echo "Selected file: $filename"
```

Type `app.log`, then press **Enter**. The message becomes `Selected file: app.log`.

- `read` stores the typed line in `filename`.
- `-p` supplies the prompt displayed before the input.
- `-r` keeps backslashes as ordinary characters.

Positional parameters

Université de Rennes
Louis Ledoux

A script uses these parameters to read its arguments:

```
bash arguments.sh "L3 MIAGE" Unix
```

Parameter	Meaning	Value for this call
`$0`	Script name	`arguments.sh`
`$1`	First argument	`L3 MIAGE`
`$2`	Second argument	`Unix`
`$#`	Number of arguments	2
`"$@"`	All arguments separately	`"L3 MIAGE" "Unix"`
`"$*"`	All arguments as one string	`"L3 MIAGE Unix"`

Passing arguments with spaces

Université de Rennes
Louis Ledoux

Use the **same script call**:

```
bash arguments.sh "L3 MIAGE" Unix
```

Inside the script, `printf` uses `%s` for a string and `\n` for a newline.
It repeats the format for each argument.

"$@": two arguments

```
printf '<%s>\n' "$@"
```

```
<L3 MIAGE>
<Unix>
```

"$*": one string

```
printf '<%s>\n' "$*"
```

```
<L3 MIAGE Unix>
```

Use "$@" to pass each argument on unchanged.

Saving a command's output

Université de Rennes
Louis Ledoux

From the extracted TP files:

```
cd ~/NUnix/files
tpfiles=$(pwd)
cd ..
ls "$tpfiles"
```

Bash runs `pwd` and stores its printed path in `tpfiles`.
That **absolute path** still works after `cd ...`

Running commands in another Bash

Université de Rennes
Louis Ledoux

```
bash -c 'echo "Hello from another Bash"'
```

- `-c` tells Bash to execute the **following text as commands**.
- The current shell starts a **child Bash**, then waits for it to finish.
- Single quotes pass the text unchanged for that child to interpret.

`bash script.sh` reads commands from a file.
`bash -c 'commands'` reads them from the next argument.

Sharing a variable with a script

Université de Rennes
Louis Ledoux

```
tpfiles="$HOME/NUnix/files"
export tpfiles
bash -c 'echo "$tpfiles"'
```

- The assignment stores the path in the **current shell**.
- `export` includes it in the environment passed to newly started programs.
- The child Bash can now print the same path.

A child receives a **copy**. Changing it does not change the parent's variable.

2. Tests and decisions

Before reading a file, how can a script check that it exists and is readable?

Testing a condition

Université de Rennes
Louis Ledoux

Does the directory `logs` exist? These two commands ask the same question:

```
test -d logs
[ -d logs ]
```

`-d` means **is a directory**. `[ ... ]` is another way to write `test`.

A test returns a status; it prints no answer.
`0`: the condition is true. `1`: the condition is false.

Comparing integers

Université de Rennes
Louis Ledoux

Compare **integers** with these operators inside `[ ... ]`:

Operator	Read it as	Mathematical symbol
`-eq`	Equal to	=
`-ne`	Not equal to	≠
`-lt`	Less than	<
`-le`	Less than or equal to	≤
`-gt`	Greater than	>
`-ge`	Greater than or equal to	≥

Read `[ "$count" -gt 10 ]` as **count is greater than 10**.

Testing text

Université de Rennes
Louis Ledoux

An interactive script can stop when the user types end.

Test	True when...
`[ "$answer" = "end" ]`	The answer is end
`[ "$answer" != "end" ]`	The answer is different from end
`[ -z "$answer" ]`	The answer is empty
`[ -n "$answer" ]`	The answer is not empty

Use = for text and `-eq` for integer comparisons.

Testing files and permissions

Université de Rennes
Louis Ledoux

A script can check a path before using it:

Test	True when...
`[ -e "$filename" ]`	The path exists
`[ -f "$filename" ]`	It is a regular file
`[ -d "$filename" ]`	It is a directory
`[ -r "$filename" ]`	Reading is allowed
`[ -w "$filename" ]`	Writing is allowed
`[ -x "$filename" ]`	Executing / directory traversal is allowed

if and else

Université de Rennes
Louis Ledoux

```
count=12
if [ "$count" -gt 10 ]; then
    echo "More than ten"
else
    echo "Ten or fewer"
fi
```

A true test runs the `then` branch; a false test runs `else`. `fi` marks the end.

Spaces are required after `[` and before `]`.
Correct: `[ "$count" -gt 10 ]`
Incorrect: `["$count" -gt 10]`

More than two cases: elif

Université de Rennes
Louis Ledoux

```
count=10
if [ "$count" -gt 10 ]; then
    echo "More than ten"
elif [ "$count" -eq 10 ]; then
    echo "Exactly ten"
else
    echo "Fewer than ten"
fi
```

`elif` means **else if**. Try each test in order and run the first true branch.
Here, the output is `Exactly ten`.

Conditional command lists

Université de Rennes
Louis Ledoux

```
mkdir results && echo "Directory created"
test -d results || echo "Directory is missing"
```

- a && b: run b only when a succeeds.
- a || b: run b only when a fails.
- ! a: invert the success/failure result.

Three conditions with &&

Université de Rennes
Louis Ledoux

Example call: `bash check.sh report.log ERROR 10`

```
if [ "$#" -gt 2 ] &&
   [ -f "$1" ] &&
   [[ "$1" == report* ]]; then
    echo "Inputs accepted"
fi
```

1. **At least three** arguments.
2. `$1` names an **existing regular file**.
3. `$1` **starts with report**.

`[[ ... ]]` supports patterns in Bash. Keep `report*` unquoted.
&& means **AND**: stop checking as soon as one condition is false.

Checking the script's arguments

Université de Rennes
Louis Ledoux

Save as `count-lines.sh`:

```
#!/bin/bash
if [ "$#" -ne 1 ]; then
    echo "Usage: $0 FILE" >&2
    exit 2
fi
wc -l < "$1"
```

- `$#` is the argument count; `-ne 1` means **not equal to 1**.
- `>&2` sends the message to standard error.
- `exit 2` stops the script with a nonzero status.

case selects a matching pattern

Université de Rennes
Louis Ledoux

```
read -r -p "Enter a command: " choice
case "$choice" in
    all|ALL)
        ls -la ;;
    list|LIST)
        ls -l ;;
    *)
        echo "Invalid choice" ;;
esac
```

| separates alternative patterns; ;; ends a branch.
*) handles any other answer; esac ends the selection.

3. Loops and counters

How can we process several files, or ask again until the user types end?

Calculating with integers

Université de Rennes
Louis Ledoux

```
count=3
echo "$((count + 2))"
echo "$count"
```

```
5
3
```

Bash replaces `$((count + 2))` with 5 before running echo.
The calculation gives 5; `count` still contains 3.

No decimal part: `$((5 / 2))` gives 2.

Updating a counter

Université de Rennes
Louis Ledoux

```
count=3
count=$((count + 1))
echo "$count"
```

`$((count + 1))` calculates 4; the assignment stores it in `count`.

The TD also uses `expr`, which **prints** a calculation's result:

```
count=$(expr "$count" + 1)
```

`$(...)` captures that result. Here, `count` becomes 5.

for visits a list of words

Université de Rennes
Louis Ledoux

```
for group in L3info L3miage M1info; do
    echo "$group"
done
```

Each iteration assigns the next word to `group`, then runs the body.

The list can contain literal words, numbers, filenames or `"$@"`.

One iteration per script argument

Université de Rennes
Louis Ledoux

In `show-files.sh`:

```
for filename in "$@"; do
    echo "File: $filename"
done
```

Run it with:

```
bash show-files.sh notes.txt "meeting notes.txt"
```

Two iterations. `"$@"` keeps `meeting notes.txt` as one filename.

while repeats after a successful test

Université de Rennes
Louis Ledoux

```
i=1
while [ "$i" -lt 10 ]; do
    echo "$i"
    i=$((i + 1))
done
```

Test **before** each iteration.
The loop prints the integers **1 through 9**.

Asking again until the user stops

Université de Rennes
Louis Ledoux

```
read -r -p "Word (end to stop): " word
while [ "$word" != "end" ]; do
    echo "You entered: $word"
    read -r -p "Word (end to stop): " word
done
```

end makes the test false and stops the loop.
The last read updates word before the next test.

while can test a whole command list

Université de Rennes
Louis Ledoux

```
i=1
while
    echo "Checking i=$i"
    [ "$i" -le 3 ]
do
    echo "Body: $i"
    i=$((i + 1))
done
```

The **last command** before do decides whether the body runs.

The condition runs once more than the body

Université de Rennes
Louis Ledoux

```
Checking i=1
Body: 1
Checking i=2
Body: 2
Checking i=3
Body: 3
Checking i=4
```

- At **1, 2, 3**, the test succeeds.
- At **4**, the final test fails.
- The body does not run a fourth time.

4. Functions and debugging

How can we reuse a file-processing step and check a script before running it?

Defining and calling a function

Université de Rennes
Louis Ledoux

Define the function, then call it with a filename:

```
count_lines() {
    local filename="$1"
    wc -l < "$filename"
}
count_lines "names.txt"
```

- Inside the function, `$1` is **the argument passed to this call**.
- `local` gives the function its own `filename` variable.
- The result is printed on standard output, ready for a redirection or pipe.

Check syntax and trace execution

Université de Rennes
Louis Ledoux

```
bash -n count-lines.sh
bash -x count-lines.sh notes.txt
```

- `-n`: parse the script without executing its commands.
- `-x`: execute and trace expanded commands on stderr.
- Check output **and** status on ordinary and failing inputs.

5. Build a log report

Support has several logs. Can one script count matching lines in each and save a report?

The task and the files

Université de Rennes
Louis Ledoux

Support checks a web service and an overnight job. Count lines matching a pattern in **each log**. Save the summary in `search-report.txt`.

Download the example files, extract them and open `report.sh` in `report-demo`.

```
bash report.sh web.log "worker night.log"
```

- Ask for a pattern, such as ERR0R.
- Check the arguments and file access.
- Process each file, then save its count.

1. Check that files were supplied

Université de Rennes
Louis Ledoux

Start `report.sh` with:

```
#!/bin/bash
if [ "$#" -eq 0 ]; then
    echo "Usage: $0 LOGFILE..." >&2
    exit 1
fi
```

Predict: what happens with `bash report.sh` and no filename?

The script prints its usage on stderr and stops with status 1.

2. Ask what to search for

Université de Rennes
Louis Ledoux

Add after the argument check:

```
read -r -p "Pattern to count: " pattern
echo "Searching for: $pattern"
```

Run the script with the two logs and type ERR0R.

read fills `pattern`. Its value will be reused for every file.

3. Process one file

Université de Rennes
Louis Ledoux

Add a function, then call it on the first filename:

```
report_file() {
    local matches
    matches=$(grep "$pattern" "$1" | wc -l)
    echo "$1: $matches matching lines"
}
report_file "$1"
```

`grep` selects the lines; `wc -l` counts them.
`$(...)` stores that count in the function's local variable.

4. Check and process every file

Université de Rennes
Louis Ledoux

Replace the single function call with this loop:

```
for logfile in "$@"; do
    if [ -f "$logfile" ] && [ -r "$logfile" ]; then
        report_file "$logfile"
    else
        echo "Cannot read file: $logfile" >&2
        exit 1
    fi
done
```

Predict: does `worker night.log` cause one iteration or two?

5. Save the report

Université de Rennes
Louis Ledoux

Before the loop, create the report and write its heading:

```
report="search-report.txt"
echo "Matching lines for: $pattern" > "$report"
```

Inside the loop, redirect the function's output:

```
report_file "$logfile" >> "$report"
```

Why >> here? Each file's result must be added after the previous one.

After the loop: `echo "Report saved in $report"`.

6. Run it and inspect the result

Université de Rennes
Louis Ledoux

```
bash report.sh web.log "worker night.log"
```

Type ERROR, then run `cat search-report.txt`:

```
Matching lines for: ERROR
web.log: 3 matching lines
worker night.log: 2 matching lines
```

Try WARN, FATAL, a missing file and a call with no arguments.
Complete script

CM3 summary: writing a script

Université de Rennes
Louis Ledoux

1. Get inputs with `read` or the script's arguments.
2. Store values, quote them and check the inputs.
3. Use tests and loops to decide what to repeat.
4. Put a reusable task in a function.
5. Run the script with normal and incorrect inputs; inspect its output.

Further examples and TP tools

Reference for the TD/TP: extra Bash forms, worked examples and text-editing tools

Options for the TP script

Université de Rennes
Louis Ledoux

```
bash main.sh -p samples -h
```

Bash passes these as ordinary arguments. **The script interprets them** with `getopts "hp:" option`, reading one option at a time:

Item	Meaning
`h`	Accept `-h` without a value
`p:`	Accept `-p` with a required value
`option`	Variable receiving the option letter
`OPTARG`	Variable receiving its value, here `samples`

Reading options with getopts

Université de Rennes
Louis Ledoux

```
directory="."
while getopts "hp:" option; do
    case "$option" in
        h) echo "Usage: $0 [-p DIRECTORY]"; exit 0 ;;
        p) directory="$OPTARG" ;;
        *) exit 1 ;;
    esac
done
echo "Selected directory: $directory"
```

The loop runs while another option is available.
An unknown option or a missing value reaches the `*)` branch.

Removing arguments already handled

Université de Rennes
Louis Ledoux

After `bash inspect.sh -p logs app.log`, the script can do:

```
directory="$2"
shift 2
echo "$directory"
echo "$1"
```

Output: `logs`, then `app.log`.
`shift 2` removes the first two arguments and renumbers the remaining ones.

After `getopts`, `OPTIND` is the position of the next argument.
`shift "$((OPTIND - 1))"` removes the arguments it has read.

Replacing text with sed

Université de Rennes
Louis Ledoux

For a file containing `cat cat`:

```
sed 's/cat/dog/' animals.txt
sed 's/cat/dog/g' animals.txt
```

Output: `dog cat`, then `dog dog`.

- `s/old/new/` replaces the first match **on each line**.
- The final `g` replaces every match on each line.
- These commands print the result and leave the file unchanged.

`sed -i 's/cat/dog/g' animals.txt` changes the file itself.
Practise on copies.

Bash: Bourne again

Université de Rennes
Louis Ledoux

Bash = Bourne-Again SHell.

The name puns on **Stephen Bourne**, author of the earlier Unix shell `sh`.

We use Bash syntax in this course.
`sh script.sh` may select a different shell with fewer features.

The shebang: selecting an interpreter

Université de Rennes
Louis Ledoux

```
#!/bin/bash
printf 'Hello, MIAGE\n'
```

On Linux, direct execution of `./hello.sh` uses the interpreter named after `#!`.

The extension `.sh` does not choose Bash.
The first line does. We call it the **shebang**.

One symbol, several names

Université de Rennes
Louis Ledoux

Symbol	Names you may hear
#	hash, number sign; pound sign in US usage
#	French: **croisillon**, also **carré**
♯	The musical sharp sign (*dièse*)

#!: **hash** followed by **bang**.
The symbol's name and its role depend on context.

Quoted variable expansions

Université de Rennes
Louis Ledoux

```
filename="meeting notes.txt"
cat "$filename"
echo '${filename}'
echo "${filename}"
```

- Double quotes preserve the expanded value as one argument.
- Single quotes preserve the literal text `${filename}`.

Printing values

Université de Rennes
Louis Ledoux

```
name="Alice"
echo "Hello, $name"
printf 'Hello, %s\n' "$name"
```

`echo` is convenient for simple messages.
`printf` gives explicit control over formatting and newlines.

A child receives a copy

Université de Rennes
Louis Ledoux

```
export message="parent"
bash -c 'message="child"; echo "$message"'
echo "$message"
```

```
child
parent
```

The condition can be grep

Université de Rennes
Louis Ledoux

```
if grep ' ERROR ' app.log; then
    echo "At least one error was found"
else
    echo "No match, or grep could not read the file"
fi
```

`if` runs `then` when the command returns status **0**.
Any other status selects `else`. Here, `grep` succeeds when it finds a matching line.

Which messages appear?

Université de Rennes
Louis Ledoux

```
true  || echo "A"
false || echo "B"
true  && echo "C"
false && echo "D"
```

```
B
C
```

Checkpoint: command status

Université de Rennes
Louis Ledoux

1. Status `0`: true or false?
2. Does `false; echo yes` print?
3. Does `false && echo yes` print?
4. Status of `false || true`?

1. True / success.
2. Yes. `;` always continues.
3. No.
4. `0`, from the final executed command, `true`.

Read a file line by line

Université de Rennes
Louis Ledoux

```
while IFS= read -r name; do
    printf 'Hello, %s\n' "$name"
done < names.txt
```

- `IFS=` disables trimming of leading/trailing spaces for this `read`.
- `-r` keeps backslashes unchanged, as in the earlier prompt.
- `read` succeeds for each complete line; at **EOF**, it fails and the loop stops.
- `< names.txt` supplies input to the **whole loop**.

until repeats after a failed test

Université de Rennes
Louis Ledoux

```
i=1
until [ "$i" -ge 10 ]; do
    echo "$i"
    i=$((i + 1))
done
```

This also prints **1 through 9**.
The loop stops when the condition succeeds.

break, continue and exit

Université de Rennes
Louis Ledoux

Command	Effect
`continue`	Skip the rest of the current iteration
`break`	Leave the current loop
`exit n`	End the script with status `n`

After `break`, execution continues **after the loop**.

Command substitution

Université de Rennes
Louis Ledoux

```
echo "Current directory: $(pwd)"
```

The shell runs pwd and replaces `$(pwd)` with its **standard output**.

The older form is `` `pwd` ``.
We will write `$(pwd)` because nested commands are easier to read.

Newlines, semicolons and continuations

Université de Rennes
Louis Ledoux

```
echo "first"
echo "second"; echo "third"
echo "a longer message" \
     "continued on the next line"
```

- A newline or ; separates commands.
- A backslash immediately before a newline continues the line.

Integer, readonly and array variables

Université de Rennes
Louis Ledoux

```
declare -i count=3
readonly course="NUnix"
declare -a words
words[0]="one"
words[1]="two"
echo "${words[0]}"
```

`unset count` removes a variable. A readonly variable cannot be reassigned or unset in that shell.

Familiar environment variables

Université de Rennes
Louis Ledoux

Variable	Meaning
`HOME`	Home directory
`USER`	Account name, usually set at login
`PWD`, `OLDPWD`	Current and previous working directories
`PATH`	Colon-separated command search directories
`SHELL`	Configured login shell, not necessarily the current shell
`TERM`	Terminal type
`HOSTNAME`	Host name in Bash
`MANPATH`	Manual search path, when explicitly set

Arithmetic with let and expr

Université de Rennes
Louis Ledoux

```
let 'count = 3 + 5'
echo "$count"
expr 3 + 5
expr 3 \* 5
```

- `let` evaluates an arithmetic expression in Bash.
- `expr` takes separate arguments, so its spaces matter.
- Escape * so the shell passes it to `expr` literally.

Bash arithmetic shortcut: ((...))

Université de Rennes
Louis Ledoux

```
count=3
((count = count + 1))
echo "$count"
```

```
4
```

`((count = count + 1))` tells Bash to calculate and update `count`. It prints nothing; `echo` displays the new value.

Here, it does the same update as `count=$((count + 1))`.

Bash tests with double brackets

Université de Rennes
Louis Ledoux

```
filename="report.txt"
if [[ "$filename" == *.txt ]]; then
    echo "Text file"
fi
```

`[[ ... ]]` is another form of test available in Bash.
Here, `== *.txt` checks whether the name **ends in `.txt`**.

With `[ ... ]`, string equality compares literal text; this Bash form also accepts patterns.

Spaces are still required: `[[ condition ]]`.

Checkpoint: arithmetic and tests

Université de Rennes
Louis Ledoux

1. Value of `$((5 / 2))`?
2. Is `n = 3` an assignment?
3. Status of `false`?
4. Does `-f` accept every existing directory?

1. 2 (integer division).
2. No. Use n=3.
3. 1
4. No. It tests for a regular file.

Checking access to a file

Université de Rennes
Louis Ledoux

```
#!/bin/bash
read -r -p "Enter a filename: " filename
if [ ! -w "$filename" ]; then
    echo "File is not writable"
    exit 1
elif [ ! -r "$filename" ]; then
    echo "File is not readable"
    exit 1
else
    echo "File is writable and readable"
fi
```

! negates a test. `-w` checks writing; `-r` checks reading.

Comparing two files

Université de Rennes
Louis Ledoux

```
diff "$1" "$2" > diff.out
status=$?
if [ "$status" -eq 0 ]; then
    echo "Files are identical"
elif [ "$status" -eq 1 ]; then
    cat diff.out
else
    echo "Comparison failed" >&2
fi
rm -f diff.out
```

`status=$?` saves the previous command's status. For `diff`:
0: identical; **1**: different; **greater than 1**: error.

Copying a known list of files

Université de Rennes
Louis Ledoux

```
#!/bin/bash
destination=$1
for file in f1 f2 f3; do
    cp "$file" "$destination/$file"
done
```

Call with an **existing destination directory** as the first argument.

A loop can feed a pipeline

Université de Rennes
Louis Ledoux

```
for number in 6 3 1 2; do
    echo "$number"
done | sort -n
```

```
1
2
3
6
```

Repeating until interrupted

Université de Rennes
Louis Ledoux

```
while true; do
    echo "alive..."
    sleep 3
done
```

`true` always succeeds.
Use **Ctrl+C** to interrupt this foreground loop.

Skipping 4, stopping at 7

Université de Rennes
Louis Ledoux

```
for x in 1 2 3 4 5 6 7 8 9; do
    if [ "$x" -eq 4 ]; then
        continue
    fi
    echo "$x"
    if [ "$x" -eq 7 ]; then
        break
    fi
done
echo "After the loop: x=$x"
```

The loop's output

Université de Rennes
Louis Ledoux

```
1
2
3
5
6
7
After the loop: x=7
```

- At **4**, `continue` skips `echo`.
- At **7**, `echo` runs before `break`.
- The final message runs after the loop.

Function scope

Université de Rennes
Louis Ledoux

```
label="outside"
show_label() {
    local label="inside"
    echo "$label"
}
show_label
echo "$label"
```

```
inside
outside
```

The local `label` hides the outer variable during the call.
The outer value is available again when the function finishes.

Builtins: shell and process control

Université de Rennes
Louis Ledoux

Builtin	Use
`cd`, `pwd`	Change / report the current directory
`history`	Display command history
`exec`	Replace the shell with another program
`exit`	End the current shell
`kill`	Send a signal
`trap`	Arrange a response to a signal or shell event

Builtins: data and parameters

Université de Rennes
Louis Ledoux

Builtin	Use
`declare`, `readonly`, `local`	Define variable attributes or scope
`export`, `set`	Manage the environment / shell settings
`echo`, `read`	Print output / read input
`let`, `test`	Arithmetic / condition tests
`shift`	Discard leading positional parameters
`return`	End a function and report its status

Reading a small script

Université de Rennes
Louis Ledoux

```
#!/bin/bash
for filename in "$@"; do
    if [ -f "$filename" ]; then
        wc -l "$filename"
    fi
done
```

For each argument that names a regular file, print its line count.